Problem 1

Course Progression Challenge: Student Proposal

Build a live course progression platform that tracks every subject, every session and every campus through the semester: what was planned, what was actually taught, and whether each subject will finish before its exams.

1. The challenge

MSOT runs the Mirai Track and the University Track across HI-Tech, Ratnam, RGIB and JUJ. Today, syllabus delivery is tracked in spreadsheets that faculty update by hand. They are easy to forget, hard to read, and nobody can see at a glance which subject is falling behind until it is close to the exam.

Your task: build a **Course Progression Tracker** that lets faculty log each session in under a minute, automatically works out where every subject should be on any given day, and shows academic leadership (and students) exactly how each course is pacing against the academic calendar and exam schedule.

You do not need to invent logic. The current Excel tracker already defines planned hours, buffer ratio and status rules; this brief reproduces them. **The real challenge is turning that into a product** that faculty actually use every day and that stays accurate for a whole semester.

Three principles every solution must respect:

- **Effortless**: logging a session must take a faculty member under 60 seconds, from a phone.
- **Accurate**: every percentage and date on the dashboard can be traced back to the logged sessions and the calendar.
- **Early warning**: a subject that is falling behind is flagged weeks before the exam, not days.

2. What to build

Must-have (required to qualify)

1. **Semester setup (admin side)**: create a semester per campus with start/end dates, working Saturdays rule (all / none / 2nd & 4th), holidays and events, and exam windows (sessionals, mid-term, PUT, end-term). Import from CSV/Excel.
2. **Timetable setup**: enter the weekly timetable per batch: subject, faculty, day, start/end time. Planned teaching hours per weekday are derived from it automatically.
3. **Syllabus / lesson plan upload**: faculty upload units and topics with planned hours per session and mark which topics are in scope for each exam.
4. **Daily session logging (faculty side)**: for each scheduled session: status (Completed / In Progress / Skipped / Merged / Cancelled), % complete, what was covered, what was left over, carry-forward to which topic, classwork/homework status, notes. Pre-filled from the timetable so the faculty only confirms.

5. **Automatic planned dates**: for every topic, compute the date it should be taught by walking the calendar and spending each weekday's teaching hours. Holidays and cancelled classes shift every later date automatically.
6. **Progress dashboard**: per subject, per track and per campus: planned hours, covered hours, coverage %, remaining hours, available hours till the deadline, buffer ratio, status (ON TRACK / TIGHT / DANGER / COMPLETE) and pace vs plan. Rolled-up track status shows the **worst** subject, not an average.
7. **Exam readiness view**: for each upcoming exam, the % of in-scope topics covered so far and the hours still needed before the exam date.
8. **Student view**: students of a batch see what was covered in each subject, what is coming next, and the upcoming exam dates and syllabus scope.
9. **Missed-log alerts**: if a scheduled session is not logged by end of day, the faculty gets a reminder; if still missing after 48 hours, the Campus Manager is notified.

Stretch goals (bonus marks)

- Log a session by WhatsApp / Telegram message or voice note, parsed by an LLM into a structured entry the faculty confirms.
- AI weekly summary per campus: which subjects slipped, why (from notes), and suggested recovery (extra session, merge topics).
- Google Calendar / Outlook sync of exam dates and extra classes.
- Visiting faculty lecture count per month (planned vs taken) and honorarium export.
- Export of all data to CSV / Google Sheets.

3. How progress is calculated

These definitions must be used so results match MSOT's existing tracker. Teams may add metrics but must not change these.

Metric	**Definition**
Hours in scope	Sum of planned hours of all topics, excluding those marked Skipped.
Covered hours	Completed topics count fully; In Progress topics count planned hours × % complete.
Coverage %	Covered hours ÷ hours in scope.
Available hours	Teaching hours left on the calendar between today and the subject's target finish date (timetable hours on working days only).
Buffer ratio	Available hours ÷ remaining hours. ≥ 1.15 ON TRACK · 1.00–1.15 TIGHT · < 1.00 DANGER (thresholds configurable).

Pace vs plan	Covered hours − hours that should have been covered by today. Negative = behind.
Overdue	A topic whose planned date has passed and is not fully covered.

Hours, not topic counts: sessions at MSOT run 50 minutes to 2 hours, so every percentage must be weighted by planned hours.

4. Users and permissions

Role	**Can do**
Admin / Program Manager	Set up campuses, semesters, calendars, exam windows; view all dashboards; export.
Campus Manager	Manage timetable and faculty for their campus; view campus dashboards; receive alerts.
Faculty (incl. visiting)	Upload lesson plan; log sessions for their own subjects only; see their own subject dashboard.
Student	Read-only view of their batch: topics covered, next topics, exam schedule and scope.

5. Suggested stack

Need	**Options**
App type	Web app (responsive, mobile-first) or PWA; optional mobile app
Backend	Node.js / Python (FastAPI, Django) with a REST or GraphQL API
Database	PostgreSQL (Supabase, Neon) or SQLite for local mode
Auth	Google Workspace login for staff (msot.org), OTP / ID login for students
Charts	Recharts, Chart.js, Apache ECharts
Notifications	Email (SMTP), WhatsApp Business API, Telegram Bot API
Import / export	SheetJS, pandas, Google Sheets API

LLM (stretch)	Hosted API (OpenAI, Claude, Gemini, Azure) or local via Ollama

6. Testing requirements

Every submission must include a **Test Report**. Submissions without one are not evaluated.

Test area	**Minimum evidence**
Calculation accuracy	For 3 subjects, dashboard figures match a hand-calculated spreadsheet to 2 decimal places
Calendar logic	Adding a holiday or cancelling a class shifts every later planned date correctly; 2nd/4th Saturday rule works for 3 months
Logging speed	5 faculty volunteers log a session; median time under 60 seconds on a phone
Alerts	Unlogged session triggers faculty reminder and CM escalation at the right times
Permissions	Faculty cannot edit another faculty's subject; students cannot edit anything
Exam readiness	In-scope coverage and hours-needed shown correctly for mid-term and end-term
Data import	A full semester (4 campuses, 25+ subjects) imports from CSV without manual fixes

7. Scalability requirement

- Support **4 campuses, 60+ faculty, 30+ subjects and 1,000+ students** in one deployment.
- Handle the end-of-day spike: **all faculty logging within the same 15 minutes** with p95 response under 1 second.
- Dashboards for a full semester load in under 2 seconds.
- Submit a load test (e.g. k6 / Locust) at 50, 100 and 200 concurrent users with p95 latency and error rate.

8. Deliverables and judging

Submission checklist

- ☐ GitHub repo with README and one-command setup (e.g. Docker Compose)
- ☐ Live demo link or 5-minute demo video
- ☐ Architecture diagram and design document (≤3 pages)

- ☐ Sample semester data loaded: one campus, one batch, full calendar and timetable
- ☐ Test Report and load test results

Judging criteria

Criterion	**Weight**
Accuracy of calculations and calendar logic	25%
Faculty logging experience (speed, ease, mobile)	20%
Dashboard clarity and early-warning usefulness	20%
Exam readiness and student view	10%
Testing depth and scalability	15%
Code quality and documentation	10%

9. What you get

- A complete product for your resume, with a live demo and GitHub repo.
- If selected, your product is **used at MSOT** to track syllabus delivery across all campuses.
- Prize: to be announced by MSOT. Awarded when the solution passes the pilot on one campus for at least 4 weeks.

Problem 2

Gamified Learning Challenge: Student Proposal

Build a platform that turns everyday learning at MSOT into a game: points, streaks, quests, leagues and team battles tied to real classes, assignments and coding practice, so studying feels like entertainment rather than an obligation.

1. The challenge

Students learn more when they want to come back every day. Today, classwork, homework, bi-weekly tests and coding practice feel like separate chores with no visible progress until marks are released. Engagement drops after the first few weeks of the semester and again after mid-terms.

Your task: build a **Gamified Learning Platform** that rewards the behaviours that actually lead to learning (showing up, practising, revising, helping peers) and makes progress visible every day, without turning learning into pure competition or rewarding shortcuts.

The real challenge is game design, not graphics: picking rewards that drive real learning, keeping it fair for every student, and making it fun enough that students open it without being told to.

Three principles every solution must respect:

- **Learning first**: every point earned must map to a real learning action. No points for idle clicks.
- **Fair and inclusive**: a student who is behind must still have a way to win (personal bests, leagues, improvement badges), not just the top 10.
- **Separate from grades**: game points never change official marks; official marks may feed in, but never the other way round.

2. What to build

Must-have (required to qualify)

1. **Student profile and XP**: each student has a level, XP total, and a history of how every point was earned.
2. **XP rules engine (admin configurable)**: points for attendance, classwork/homework completion, bi-weekly test scores, daily quiz, coding practice and club participation. Admin can change values without code changes.
3. **Daily quiz / micro-challenge**: 3–5 questions per day on topics actually taught that week (from the course syllabus). Question bank uploaded by faculty or AI-generated and approved by faculty.
4. **Streaks**: daily learning streaks with a limited number of "freeze" days so one bad day does not wipe out progress.
5. **Badges and quests**: topic quests (e.g. "Finish all Recursion problems"), milestone badges, and improvement badges (e.g. "+15% on last test").

6. **Leagues and leaderboards**: weekly leagues of ~20 students at similar levels with promotion/relegation, plus batch and campus boards. Students can choose to appear anonymously.
7. **Team battles**: section vs section or campus vs campus events (e.g. a weekend coding sprint) with a live scoreboard.
8. **Faculty dashboard**: see class engagement, which topics students struggle with in quizzes, and who has gone quiet for 5+ days.
9. **Anti-gaming checks**: rate limits on quizzes, detection of copy-pasted answers and answer-sharing patterns, and admin audit of unusual XP spikes.

Stretch goals (bonus marks)

- Integration with LeetCode / Codeforces / GitHub to award XP for verified practice.
- Boss battles: a timed revision challenge a week before mid-term and end-term exams.
- Rewards store: redeem XP for non-academic perks MSOT approves (event passes, merch, mentor sessions).
- AI study buddy that explains wrong quiz answers without giving away future answers.
- Hindi / Hinglish support in quiz explanations.

3. Points economy (starting point)

Teams may change these values but must explain why and show that the economy cannot be farmed.

Action	**Suggested XP**	**Limit**
Attend a class	10	Per class, from official attendance only
Submit classwork / homework on time	20	Per assignment
Daily quiz completed	15 + 5 per correct answer	Once per day
Bi-weekly test score	Score % ÷ 2	Per test
Solve a verified coding problem	10 / 20 / 40 (easy / medium / hard)	Max 10 per day
Club activity participation	25	Per verified club day
7-day streak	50 bonus	Once per streak

Help a peer (answer accepted on discussion board)	15	Max 3 per day

4. Game design rules

- Every source of XP must be **verifiable** from MSOT data (attendance, submissions, test marks, platform APIs) or the platform itself.
- No negative points for absence; missing out is its own cost. Avoid mechanics that shame students publicly.
- Leaderboards reset weekly or monthly so new students and late starters can compete.
- Students can opt out of public leaderboards without losing access to any other feature.
- Faculty override always wins: they can void suspicious XP with a logged reason.

5. Suggested stack

Need	**Options**
App type	Mobile-first web app / PWA; optional React Native or Flutter app
Backend	Node.js / Python with an event-based XP engine (rules stored in DB)
Real-time scoreboards	WebSockets, Supabase Realtime, Firebase
Database	PostgreSQL; Redis for leaderboards and streak counters
Quiz generation	Hosted LLM API or local via Ollama, with faculty approval queue
Coding profiles	LeetCode / Codeforces public APIs, GitHub REST API
Notifications	Push notifications, email, WhatsApp / Telegram bot

6. Testing requirements

Every submission must include a **Test Report**. Submissions without one are not evaluated.

Test area	**Minimum evidence**
XP correctness	Scripted scenarios for 10 students: XP totals match expected values exactly

Anti-gaming	At least 5 cheating attempts (quiz spam, shared answers, fake practice) and how each is caught
Quiz quality	50 generated questions reviewed by a faculty member; % accepted without edits
Engagement pilot	One batch uses the app for 2 weeks: daily active users, quiz completion rate, streak lengths, student feedback survey
Fairness	Show that a bottom-quartile student can still earn promotions and badges in a week
Privacy	Anonymous mode hides identity on every leaderboard and export

7. Scalability requirement

- Support **1,000+ students across 4 campuses** with live leaderboards.
- Handle a **team battle with 300 students submitting at once**: scoreboard updates within 2 seconds, no lost submissions.
- Submit a load test at 100, 300 and 500 concurrent users with p95 latency and error rate.

8. Deliverables and judging

Submission checklist

- ☐ GitHub repo with README and one-command setup
- ☐ Live demo link or 5-minute demo video
- ☐ Game design document (≤3 pages): points economy, anti-gaming rules, fairness reasoning
- ☐ Sample quiz bank for 2 subjects
- ☐ Test Report, engagement pilot results and load test results

Judging criteria

Criterion	**Weight**
Game design: does it drive real learning?	25%
Fun and student experience	20%
Fairness, inclusivity and anti-gaming	20%
Faculty insights dashboard	10%
Testing depth, pilot evidence and scalability	15%

Code quality and documentation	10%

9. What you get

- A complete product for your resume, with a live demo, GitHub repo and real engagement data.
- If selected, your product is **used by MSOT students** across all campuses.
- Prize: to be announced by MSOT. Awarded when the solution shows a measurable engagement improvement in a one-month pilot.

Problem 3

Club Activities Tracker Challenge: Student Proposal

Build one platform to run every MSOT club: membership, Club Day planning, attendance, submissions, evaluation and certificates, so that every student's co-curricular record is complete, verified and ready for their resume.

1. The challenge

The MSOT academic calendar has a Club Day almost every Saturday (PPT presentations and assignment-based activities), plus student events such as Gambade, the Hackathon, the International Conference and Cross Road. Today, club membership, attendance and submissions are scattered across WhatsApp groups, forms and spreadsheets. Coordinators cannot see who participated, and students have no verified record of what they did.

Your task: build a **Club Activities Tracker** that runs the full club cycle from enrolment to certificate, for every club on every campus, and gives MSOT one clear view of co-curricular participation.

Three principles every solution must respect:

- **Verified**: every participation record is backed by a check-in, a submission or a coordinator's approval.
- **Low effort for coordinators**: student coordinators run a Club Day from their phone in a few taps.
- **Student-owned record**: every student can download a verified co-curricular transcript at any time.

2. What to build

Must-have (required to qualify)

1. **Club registry (admin side)**: create clubs per campus with description, faculty advisor, student coordinators, capacity and meeting rules.
2. **Student enrolment**: students browse clubs and request to join (configurable limit, e.g. one primary and one secondary club); coordinators approve.
3. **Club Day calendar**: Club Days imported from the academic calendar; each club plans its activity for each Club Day (topic, format, deliverable) and gets advisor approval.
4. **Attendance check-in**: QR code or one-time code shown at the session; students check in from their phone; coordinators can correct with a reason.
5. **Submissions**: students upload PPTs, assignments, links or photos against each activity, with deadlines.
6. **Evaluation**: coordinators / advisors score submissions on a simple rubric (e.g. content, presentation, participation) with comments.
7. **Events and registrations**: create campus or inter-campus events (Gambade, Hackathon, Cross Road), with registration, team formation, check-in and results.

8. **Certificates**: auto-generate participation and achievement certificates (PDF) with a verification link / QR.
9. **Dashboards**: participation by student, club and campus; inactive clubs; students with zero participation; top contributors; exportable reports for academic reviews.

Stretch goals (bonus marks)

- Co-curricular transcript that plugs into the Placement Readiness Copilot (Problem 2) as the "Achievements" input.
- Club budget and expense requests with approval workflow.
- Duty Leave request generated automatically for approved inter-college events, ready for Campus Manager approval.
- Photo / media gallery per event with consent controls.
- WhatsApp / Telegram announcements to club members.

3. The Club Day flow

Every Club Day follows the same flow so participation is comparable across clubs and campuses. Teams may adjust timings but must justify changes.

When	Step	What the system does
7 days before	Plan	Coordinator submits the activity plan; advisor approves or requests changes.
3 days before	Announce	Members notified with topic, venue, deliverable and deadline.
On the day, start	Check-in	QR / code check-in opens for a limited window; late check-ins flagged.
During	Run	Coordinator marks presenters and any on-the-spot activity results.
Within 48 hours	Submit	Students upload deliverables; reminders sent to those who have not.
Within 5 days	Evaluate	Coordinator / advisor scores submissions; scores visible to students.
End of month	Report	Automatic club report to Campus Manager: attendance %, submissions, average scores, highlights.

4. Users and permissions

Role	Can do

Admin / Program Manager	Create clubs and events across campuses; view all dashboards; export reports.
Campus Manager	Approve clubs for their campus; view campus participation; approve Duty Leave for events.
Faculty advisor	Approve activity plans; evaluate submissions; issue achievement certificates.
Student coordinator	Plan activities, run check-in, correct attendance with reason, evaluate submissions.
Student	Join clubs, check in, submit work, view scores, download certificates and transcript.

5. Suggested stack

Need	**Options**
App type	Mobile-first web app / PWA
Backend	Node.js / Python (FastAPI, Django)
Database and files	PostgreSQL; Supabase Storage, S3 or MinIO for uploads
QR check-in	Rotating QR codes (time-based) to prevent sharing screenshots
Certificates	HTML-to-PDF (Playwright, WeasyPrint) with a public verification page
Auth	Google Workspace login for staff, OTP / ID login for students
Notifications	Email, push, WhatsApp / Telegram bot

6. Testing requirements

Every submission must include a **Test Report**. Submissions without one are not evaluated.

Test area	**Minimum evidence**
End-to-end Club Day	Full flow for 3 clubs: plan → approve → check-in → submit → evaluate → report

Check-in integrity	Screenshot sharing, late check-in and check-in from outside campus (if geo-check used): what happens in each case
Certificates	20 certificates generated; each verification link opens the correct record; tampered PDF fails verification
Events	Event with 200 registrations and team formation runs without duplicates or lost entries
Permissions	Coordinators see only their own club; students cannot edit attendance or scores
Reports	Monthly report figures match the raw attendance and submission data

7. Scalability requirement

- Support **4 campuses, 30+ clubs and 1,000+ students**.
- Handle **500 students checking in within the same 5 minutes** on a Saturday morning with p95 response under 1 second.
- Submit a load test at 100, 300 and 500 concurrent check-ins with p95 latency and error rate.

8. Deliverables and judging

Submission checklist

- ☐ GitHub repo with README and one-command setup
- ☐ Live demo link or 5-minute demo video
- ☐ Architecture diagram and design document (≤3 pages)
- ☐ Sample data: 3 clubs, one month of Club Days, one event
- ☐ Sample certificate and co-curricular transcript
- ☐ Test Report and load test results

Judging criteria

Criterion	**Weight**
Completeness of the club cycle (enrolment to certificate)	25%
Coordinator and student experience	20%
Verification and data integrity	20%

Dashboards and reports	10%
Testing depth and scalability	15%
Code quality and documentation	10%

9. What you get

- A complete product for your resume, with a live demo and GitHub repo.
- If selected, your product is **used by every MSOT club** on all campuses.
- Prize: to be announced by MSOT. Awarded when the solution runs Club Days on one campus for a full month.